ass7

April 6, 2025

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, export_graphviz
from sklearn.metrics import accuracy_score, classification_report,␣
↪confusion_matrix

import graphviz
```

[2]:
```
# Load the dataset
df = pd.read_csv("C:/Users/cools/Desktop/ML ass/drug.csv")

# Display the first 5 rows
print(df.head())
```

```
   Age Sex      BP Cholesterol  Na_to_K   Drug
0   23   F    HIGH        HIGH   25.355  drugY
1   47   M     LOW        HIGH   13.093  drugC
2   47   M     LOW        HIGH   10.114  drugC
3   28   F  NORMAL        HIGH    7.798  drugX
4   61   F     LOW        HIGH   18.043  drugY
```

[3]:
```
# Check for missing values
print(df.isnull().sum())

# Convert categorical features into numerical values using Label Encoding
le_sex = LabelEncoder()
df['Sex'] = le_sex.fit_transform(df['Sex'])

le_bp = LabelEncoder()
df['BP'] = le_bp.fit_transform(df['BP'])

le_cholesterol = LabelEncoder()
df['Cholesterol'] = le_cholesterol.fit_transform(df['Cholesterol'])
```

```
le_drug = LabelEncoder()
df['Drug'] = le_drug.fit_transform(df['Drug'])  # Encode target variable
```

```
Age            0
Sex            0
BP             0
Cholesterol    0
Na_to_K        0
Drug           0
dtype: int64
```

[4]:
```
# Define feature variables (X) and target variable (y)
X = df.drop(columns=['Drug'])  # Features: Age, Sex, BP, Cholesterol, Na_to_K
y = df['Drug']  # Target: Drug category
```

[5]:
```
# Split into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=42)

# Print dataset shapes
print(f"Training Data Shape: {X_train.shape}")
print(f"Testing Data Shape: {X_test.shape}")
```

```
Training Data Shape: (160, 5)
Testing Data Shape: (40, 5)
```

[6]:
```
# Create Decision Tree classifier
dt = DecisionTreeClassifier(criterion="entropy", max_depth=4, random_state=42)

# Train the model
dt.fit(X_train, y_train)
```

[6]:
```
DecisionTreeClassifier(criterion='entropy', max_depth=4, random_state=42)
```

[7]:
```
# Predict on test data
y_pred = dt.predict(X_test)
```

[8]:
```
# Calculate Accuracy
accuracy = accuracy_score(y_test, y_pred) * 100
print(f"Accuracy: {accuracy:.2f}%")

# Generate Classification Report
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
```

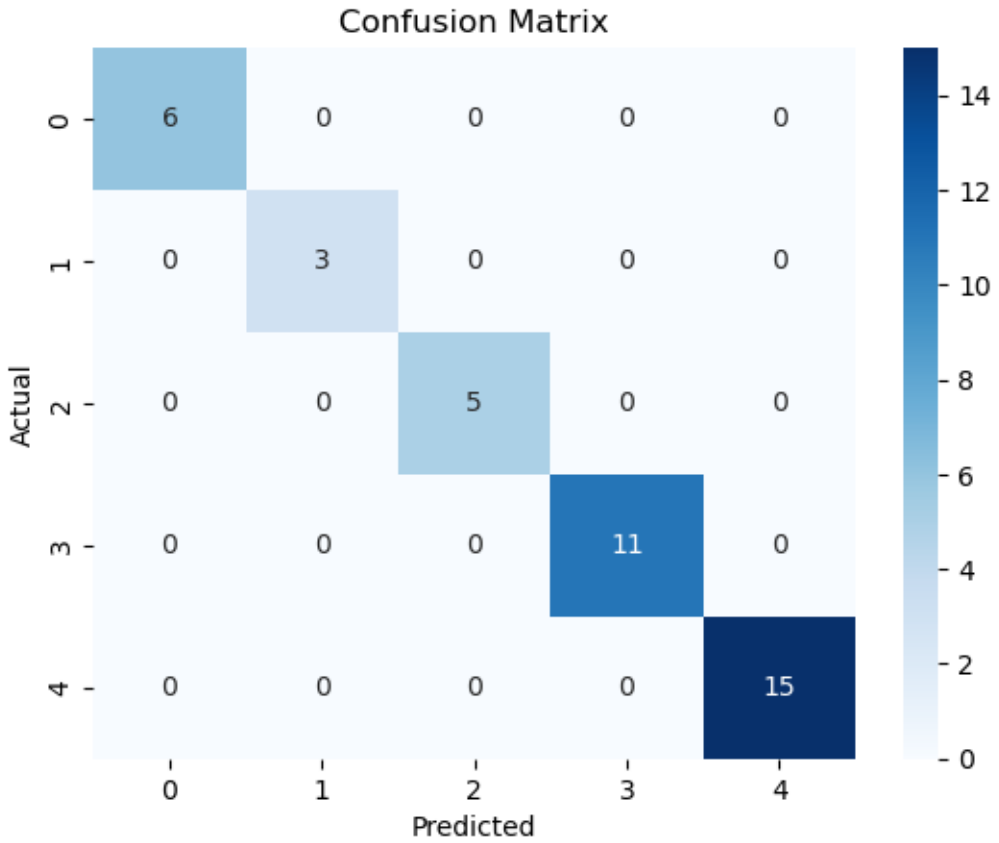
```
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

```
Accuracy: 100.00%

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         6
           1       1.00      1.00      1.00         3
           2       1.00      1.00      1.00         5
           3       1.00      1.00      1.00        11
           4       1.00      1.00      1.00        15

    accuracy                           1.00        40
   macro avg       1.00      1.00      1.00        40
weighted avg       1.00      1.00      1.00        40
```



[ ]:
```
!pip install graphviz
```

[ ]:
```
# Visualizing the Decision Tree
dot_data = export_graphviz(dt, out_file=None,
                           feature_names=X.columns,
                           class_names=le_drug.classes_,
                           filled=True, rounded=True,
                           special_characters=True)

graph = graphviz.Source(dot_data)
graph.render("decision_tree")  # Saves as a file
graph.view()  # Opens the visualization
```

[ ]:
```

```